Five Building Blocks of Robot Vision: Color Spaces, Gaussian Pyramids, Optical Flow, GrabCut, and Supervised Learning

Matthew Crist

ENG 6010 – AI in Robotics, First Test, Question 1 | October 2026

Abstract

This paper reviews five methods that sit underneath most robot perception pipelines: color spaces, Gaussian pyramids, optical flow, GrabCut segmentation, and supervised learning. For each one I summarize the idea from the original papers, the core math, and its main weakness, and I back it up with small experiments on synthetic Unitree Go2 camera and joint data from the MuJoCo simulator, where the true answer is known exactly.

1. Introduction

A legged robot like the Unitree Go2 has two streams of data: images from its front camera and numbers from its joints, feet, and IMU. Turning them into decisions takes a chain of methods. The robot first has to represent color in a way that survives changing light, look at the scene at several scales, measure how things move between frames, separate objects from the background, and finally learn a mapping from sensor data to labels such as "obstacle" or "trotting." The five topics in this review follow that chain, and each builds on the one before it: the pyramid is what makes optical flow work for large motions, and segmentation and flow produce the features and labels that supervised learning needs.

2. Review of the Methods

2.1 Color Space

RGB mixes two things a robot usually wants separated: *what color* a surface is and *how much light* hits it. If the lights dim, all three values drop together and a fixed RGB threshold fails [1]. Smith's HSV transform [2] re-expresses RGB as hue (the angle around the color wheel), saturation, and value (brightness). Because uniform dimming scales R, G, and B by the same factor, the hue stays the same. CIE $L^*a^*b^*$ [3] takes a different approach: it is designed to be perceptually uniform, with L^* for lightness and a^* (green–red) and b^* (blue–yellow) for chroma.

The tradeoff is that no space is invariant to everything. In my experiment I tuned color thresholds on a normally lit synthetic Go2 image and tested them under dim and warm light. HSV kept a mean IoU of 0.99 under dim light but fell to 0.26 under warm (yellow-tinted) light, because a tint really does shift hue. Lab a^*b^* did the opposite (0.95 warm, 0.18 dim), since chroma shrinks with brightness. Plain RGB collapsed to 0.00 in dim light. The lesson is to pick the space that matches the expected lighting change, and to add white balancing.

2.2 Gaussian Pyramid

Many vision problems need the image at several scales: a large motion or object at full resolution is a small one at low resolution. Burt and Adelson [4] built the Gaussian pyramid by repeatedly blurring with a small Gaussian-like kernel w and downsampling by two, $G_{k+1} = \downarrow_2 (w * G_k)$. The blur matters: shrinking without it causes aliasing, where fine textures turn into false patterns. They also defined the Laplacian pyramid, $L_k = G_k - \uparrow_2 G_{k+1}$, which stores only the detail lost between levels. Because each level can be added back, the original image can be rebuilt exactly. The same idea underlies scale-space theory [5], where an image is studied across a continuous range of blur levels.

In my experiments the 1/8-size level of a Go2 frame was 3 dB closer to an area-averaged reference than naive pixel skipping (31.5 vs. 28.4 dB PSNR), and the Laplacian pyramid rebuilt the image with zero error. Template matching on level 2 ran about 18 times faster than at full resolution and found the same location.

2.3 Optical Flow

Optical flow is the apparent motion of each pixel between two frames. Almost every method starts from brightness constancy: a point keeps its brightness as it moves, $I(x+u, y+v, t+1) = I(x, y, t)$. A first-order Taylor expansion gives

$$I_x u + I_y v + I_t = 0, \tag{1}$$

one equation with two unknowns. This is the *aperture problem*: along a straight edge, only the motion perpendicular to the edge can be measured. Horn and Schunck [6] solved it globally by adding a smoothness term over the whole image. Lucas and Kanade [7] solved it locally by assuming all pixels in a small window move together, which gives the least-squares system

$$\begin{bmatrix} \sum I_x^2 & \sum I_x I_y \\ \sum I_x I_y & \sum I_y^2 \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix} = - \begin{bmatrix} \sum I_x I_t \\ \sum I_y I_t \end{bmatrix}. \tag{2}$$

The matrix is only well conditioned at corners, which is why Shi and Tomasi [8] choose the points to track by its smaller eigenvalue. Because the Taylor expansion only holds for small motions, Bouguet's pyramidal version [9] runs Lucas–Kanade from the coarsest pyramid level down, so large motions become small ones. Dense methods estimate flow at every pixel: Farnebäck [10] fits a quadratic polynomial to each neighborhood and compares the polynomials between frames, and DIS [11] uses fast inverse-compositional patch search with variational refinement. Methods are usually graded by average end-point error (EPE) against ground truth [12].

Using exact ground-truth flow from the simulator's depth and camera pose, with about 12 px of motion per frame, DIS reached 1.2 px EPE and Lucas–Kanade a 0.3 px median error at corners. For larger motion (26 px), the pyramid cut Lucas–Kanade's median error from 2.0 to 0.6 px. Farnebäck was the surprise: on the raw frames it barely detected motion, and I traced this to low local contrast in the rendered texture. After histogram equalization and more pyramid levels its error dropped to 6.0 px. Flow methods depend heavily on texture and contrast, which matters for a robot looking at smooth lab floors.

2.4 GrabCut

Segmentation separates an object from its background. Boykov and Jolly [13] framed this as minimizing an energy over pixel labels $\alpha_i \in \{0, 1\}$,

$$E(\alpha) = \sum_i U(\alpha_i, z_i) + \lambda \sum_{(i,j)\in\mathcal{N}} [\alpha_i \neq \alpha_j]\, e^{-\beta \|z_i - z_j\|^2}, \tag{3}$$

where the data term U measures how well a pixel's color z_i fits the foreground or background model, and the smoothness term charges a cost for cutting between similar neighbors. A graph min-cut minimizes it exactly. GrabCut [14] made the approach practical. The user only draws a rectangle, the foreground and background are each modeled by a Gaussian mixture of colors, and the method iterates: label the pixels with a graph cut, refit the mixtures, and repeat. The user can add a few strokes to fix mistakes.

On my synthetic data, a box around the Go2 robot covered the robot with an IoU of only 0.31, while GrabCut raised it to 0.94 across three poses and two camera views. On the solid-colored objects it reached 0.98–0.99. On the hardest image, three small hint strokes improved IoU from 0.90 to 0.96. GrabCut's weakness is its reliance on color (a gray robot on a gray floor is far harder than a red box) and its need for a box from a person or a detector.

2.5 Supervised Learning

Supervised learning fits a function f that maps an input x (an image or a window of sensor data) to a label y, using labeled examples $\{(x_i, y_i)\}$. The model is chosen to minimize a loss on the training data while still

generalizing to new data [15]. Classic models include logistic regression (linear decision boundaries) and random forests [16], which average many decision trees trained on random subsets of data and features. Deep convolutional networks [17] learn features directly from pixels and dominate image tasks, but need far more labeled data. The biggest practical danger is evaluating on data that is not truly new. Kaufman et al. [18] call this *leakage*: information about the test set sneaks into training and inflates the score.

I trained classifiers to recognize five Go2 behaviors (standing, slow trot, fast trot, sway, and lying) from 1-second windows of joint, foot-force, and IMU data simulated in MuJoCo [19]. With training and test sets split by recording episode, a random forest reached 97.2% accuracy and logistic regression 96.7%. When I split the IMU windows randomly instead of by episode, accuracy jumped from 83.1% to 98.2%. Windows from the same run were landing in both sets, which is exactly the leakage problem.

3. Discussion and Conclusion

These five methods work best together. On the Go2, a color space suited to the lighting gives cleaner pixels, the pyramid makes optical flow robust to the fast motion of a walking robot, and GrabCut masks and flow fields become training data for supervised models. Each method also has a clear failure mode: color under changing light, flow on low-texture surfaces, GrabCut on low-contrast objects, and learning under data leakage. My experiments used simulated data, which gave me exact ground truth but less noise and texture than a real camera, as the Farnebäck result showed. The next step is to repeat these tests on images and joint logs recorded from the real Go2. Reading the original papers and then testing the methods against known ground truth showed me that understanding the failure modes matters as much as knowing the equations.

References

[1] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed. Pearson, 2018.

[2] A. R. Smith, “Color gamut transform pairs,” *Computer Graphics (Proc. SIGGRAPH)*, vol. 12, no. 3, pp. 12–19, 1978.

[3] CIE, *Colorimetry*, 3rd ed., CIE Publication 15:2004. Vienna: Commission Internationale de l’Éclairage, 2004.

[4] P. J. Burt and E. H. Adelson, “The Laplacian pyramid as a compact image code,” *IEEE Trans. Communications*, vol. 31, no. 4, pp. 532–540, 1983.

[5] T. Lindeberg, “Scale-space theory: A basic tool for analyzing structures at different scales,” *J. Applied Statistics*, vol. 21, no. 1–2, pp. 225–270, 1994.

[6] B. K. P. Horn and B. G. Schunck, “Determining optical flow,” *Artificial Intelligence*, vol. 17, no. 1–3, pp. 185–203, 1981.

[7] B. D. Lucas and T. Kanade, “An iterative image registration technique with an application to stereo vision,” in *Proc. IJCAI*, 1981, pp. 674–679.

[8] J. Shi and C. Tomasi, “Good features to track,” in *Proc. IEEE CVPR*, 1994, pp. 593–600.

[9] J.-Y. Bouguet, “Pyramidal implementation of the Lucas Kanade feature tracker: Description of the algorithm,” Intel Corporation, Microprocessor Research Labs, Tech. Rep., 2000.

[10] G. Farnebäck, “Two-frame motion estimation based on polynomial expansion,” in *Proc. Scandinavian Conf. Image Analysis (SCIA)*, LNCS 2749, 2003, pp. 363–370.

[11] T. Kroeger, R. Timofte, D. Dai, and L. Van Gool, “Fast optical flow using dense inverse search,” in *Proc. ECCV*, 2016, pp. 471–488.

[12] S. Baker, D. Scharstein, J. P. Lewis, S. Roth, M. J. Black, and R. Szeliski, “A database and evaluation methodology for optical flow,” *Int. J. Computer Vision*, vol. 92, no. 1, pp. 1–31, 2011.

[13] Y. Y. Boykov and M.-P. Jolly, “Interactive graph cuts for optimal boundary & region segmentation of objects in N-D images,” in *Proc. IEEE ICCV*, 2001, pp. 105–112.

[14] C. Rother, V. Kolmogorov, and A. Blake, “‘GrabCut’: Interactive foreground extraction using iterated graph cuts,” *ACM Trans. Graphics (Proc. SIGGRAPH)*, vol. 23, no. 3, pp. 309–314, 2004.

[15] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning*, 2nd ed. Springer, 2009.

[16] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[17] Y. LeCun, Y. Bengio, and G. Hinton, “Deep learning,” *Nature*, vol. 521, pp. 436–444, 2015.

[18] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, “Leakage in data mining: Formulation, detection, and avoidance,” *ACM Trans. Knowledge Discovery from Data*, vol. 6, no. 4, art. 15, 2012.

[19] E. Todorov, T. Erez, and Y. Tassa, “MuJoCo: A physics engine for model-based control,” in *Proc. IEEE/RSJ IROS*, 2012, pp. 5026–5033.